

Chapter 17: Interfaces

By Victoria Gonda

You've learned about two Kotlin custom types: Classes and objects. There's another custom type that's quite useful: **Interfaces**.

Unlike the other custom types, interfaces aren't anything you instantiate directly. Instead, they define a blueprint of behavior that concrete types **conform** to. With an interface, you define a common set of properties and behaviors that concrete types go and implement. The primary difference between interfaces and other custom types is that interfaces themselves cannot contain state.

In this chapter, you'll learn about interfaces and see why they're central to programming in Kotlin. Open up a Kotlin file to get going.

Introducing interfaces

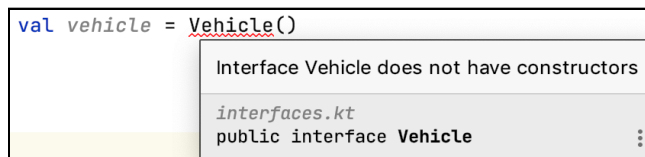
You define an interface much as you do any other custom type. Add this interface to your Kotlin file, outside the `main()` function:

```
interface Vehicle {  
    fun accelerate()  
    fun stop()  
}
```

This creates a `Vehicle` interface with blueprint instructions for two methods: `accelerate()` and `stop()`.

The keyword `interface` is followed by the name of the interface, followed by curly braces with the members of the interface inside. The big difference you'll notice is that the interface *doesn't have to contain any implementation*.

That means you can't instantiate a `Vehicle` directly:



Interface Vehicle does not have constructors

Instead, you use interfaces to enforce methods and properties on *other* types. What you've defined here is something like the *idea* of a vehicle — it's something that can accelerate and stop.

Interface syntax

An interface can be **implemented** by a class or object, and when another type implements an interface, it's required to define the methods and properties defined in the interface. Once a type implements all members of an interface, the type is said to **conform** to the interface.

Here's how you declare interface conformance for your type. Define a new class that will conform to `Vehicle`:

```
class Unicycle: Vehicle {  
    var peddling = false  
  
    override fun accelerate() {  
        peddling = true  
    }  
}
```

```

    }

    override fun stop() {
        peddling = false
    }
}

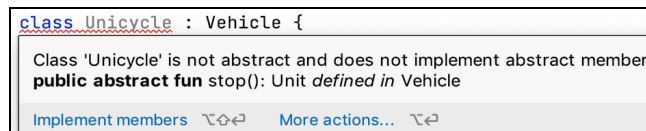
```

Here, you create a `Unicycle` that inherits from and conforms to `Vehicle`.

You follow the name of the custom type, `Unicycle`, with a colon and the name of the interface you want to conform to, `Vehicle`. This syntax might look familiar, since it's the same syntax you use to make a class inherit from another class.

Note that it *looks* like class inheritance but it isn't; objects and other custom types can also conform to interfaces with this syntax.

If you were to remove the definition of `stop()` from the class `Unicycle` above, Kotlin would display an error since `Unicycle` wouldn't have fully conformed to the `Vehicle` interfaces.



Class `Unicycle` is not abstract and does not implement abstract member `stop` defined in `Vehicle`

You'll come back to the details of implementing interfaces in a bit, but first you'll see what's possible when defining interfaces.

Methods in interfaces

In the `Vehicle` interface above, you define a pair of methods, `accelerate()` and `stop()`, that all types conforming to `Vehicle` must implement.

You declare methods on interfaces much like you would on any class or object with parameters and return values. Add `Direction` and `DirectionalVehicle`:

```

enum class Direction {
    LEFT, RIGHT
}

interface DirectionalVehicle {
    fun accelerate()
    fun stop()
    fun turn(direction: Direction)
}

```

```
fun description(): String  
}
```

The signature of these methods look the same as methods on a class, but without a body.

Methods declared in interfaces can contain default parameters, just like methods declared in classes and top-level functions. Add this class as an example:

```
interface OptionalDirectionalVehicle {  
    fun turn(direction: Direction = Direction.LEFT)  
}
```

Here, you specify a default value of `Direction.LEFT`.

When implementing such an interface, the default value of the parameter will fall through to calls on types that implement the interface. Add this class to implement your `OptionalDirectionalVehicle`:

```
class OptionalDirection : OptionalDirectionalVehicle {  
    override fun turn(direction: Direction) {  
        println(direction)  
    }  
}
```

Notice that when you implement `turn()` you don't need to repeat the default value of the `direction` parameter.

Now, try it out in your `main()` method:

```
val car = OptionalDirection()  
car.turn() // > LEFT  
car.turn(Direction.RIGHT) // > RIGHT
```

Run your code to see how the default value specified in the interface is used.

Default method implementations

Just as you can in Java 8, you can define *default implementations* for the methods in an interface.

Add some space travel to your code:

```
interface SpaceVehicle {  
    fun accelerate()  
    // 1
```

```
    fun stop() {
        println("Whoa, slow down!")
    }
}

class LightFreighter: SpaceVehicle {
    // 2
    override fun accelerate() {
        println("Proceed to hyperspace!")
    }
}
```

Note that here you:

1. Have a default implementation for `stop()` on the `SpaceVehicle` interface.
2. Don't override `stop()` in `LightFreighter`.

See what this means when you run it. Add this example to `main()`:

```
val falcon = LightFreighter()
falcon.accelerate() // > Proceed to hyperspace!
falcon.stop() // > "Whoa, slow down!"
```

Run your code. You've defined an implementation for `stop()` inside the interface, but left `accelerate()` undefined. Any types implementing `SpaceVehicle`, such as `LightFreighter` must include an implementation of `accelerate()`.

When an interface defines a default implementation, you can still override the implementation in a type that conforms to the interface. Create a `Starship` that does this:

```
class Starship: SpaceVehicle {
    override fun accelerate() {
        println("Warp factor 9 please!")
    }

    override fun stop() {
        super.stop()
        println("That kind of hurt!")
    }
}
```

Here, `Starship` overrides both of the methods declared in the `SpaceVehicle` interface, and it also uses `super` in `stop()` to call the default implementation.

Try it out:

```
val enterprise = Starship()
enterprise.accelerate()
// > Warp factor 9 please!
enterprise.stop()
// > Whoa, slow down!
// > That kind of hurt!"
```

Just as with subclasses, the super call is not required. Because you did call it, both the default implementation and your addition executes.

Properties in interfaces

You can also define properties in an interface. Add this interface:

```
interface VehicleProperties {
    val weight: Int // abstract
    val name: String
    get() = "Vehicle"
}
```

Here, you have a `VehicleProperties` with `weight` and `name`.

Interfaces cannot themselves hold state, as there are no backing fields to hold the data stored in an interface property. You must either let the property be *abstract* with no value, or give the property a default getter, like for `name` in `VehicleProperties`.

Types that implement an interface with properties can either give abstract properties a value, or provide an implementation. Add these examples:

```
class Car: VehicleProperties {
    override val weight: Int = 1000
}

class Tank: VehicleProperties {
    override val weight: Int
    get() = 10000

    override val name: String
    get() = "Tank"
}
```

Note the use of the `override` keyword on the property implementations. The `Car` class gives a value to `weight` and uses the default implementation of `name`, while the `Tank` class gives `weight` a custom getter and overrides `name`.

Interface inheritance

The `Vehicle` interface contains a set of methods that could apply to any type of vehicle, such as a bike, a car, a snowmobile or even an airplane!

You may wish to define an interface that contains all the qualities of a `Vehicle`, but that is also specific to vehicles with wheels. For this, you can have interfaces that inherit from other interfaces, similar to how you can have classes that inherit from other classes

Define a `WheeledVehicle`:

```
interface WheeledVehicle: Vehicle {  
    val numberOfWheels: Int  
    var wheelSize: Double  
}
```

Now any type you mark as conforming to the `WheeledVehicle` interface will have all of the members defined within the braces, in addition to all of the members of `Vehicle`.

Add a `Bike` that conforms to this interface:

```
class Bike: WheeledVehicle {  
    var peddling = false  
    var brakesApplied = false  
  
    override val numberOfWheels = 2  
    override var wheelSize = 622.0  
  
    override fun accelerate() {  
        peddling = true  
        brakesApplied = false  
    }  
  
    override fun stop() {  
        peddling = false  
        brakesApplied = true  
    }  
}
```

As with subclassing, any type you mark as a `WheeledVehicle` will have an **is-a** relationship with the interface `Vehicle`. The class `Bike` implements all the methods and properties defined in both `Vehicle` and `WheeledVehicle`. If any of them weren't defined, you'd receive a build error.

Defining an interface guarantees any type that conforms to the interface will have *all* the members you've defined in the interface and its parent interfaces, if any.

Mini-exercises

1. Create an interface `Area` that defines a read-only property `area` of type `Double`.
2. Implement `Area` with classes representing `Square`, `Triangle`, and `Circle`.
3. Add a circle, a square, and a triangle to an array. Convert the array of shapes to an array of areas using `map`.

Look at the challenge solutions for this chapter to check your work.

Implementing multiple interfaces

One class can only inherit from another single class. This is the property of **single inheritance**. In contrast, a class can adopt as many interfaces as you'd like!

Suppose that instead of creating a `WheeledVehicle` interface that inherits from `Vehicle`, you made `Wheeled` its own interface. Add this example:

```
interface Wheeled {  
    val numberOfWheels: Int  
    val wheelSize: Double  
}  
  
class Tricycle: Wheeled, Vehicle {  
    // Implement both Vehicle and Wheeled  
}
```

Interfaces support **multiple conformance**, so you can apply any number of interfaces to types you define. In the example above, the `Tricycle` class now has to implement all members defined in the `Vehicle` and `Wheeled` interfaces.

Interfaces in the standard library

The Kotlin standard library uses interfaces extensively in ways that may surprise you. Understanding the roles interfaces play in Kotlin can help you write clean, decoupled “Kotliny” code.

This section gives two examples of common interfaces in the standard library.

Iterator

Kotlin lists, maps, and other collection types all provide access to `Iterator` instances. `Iterator` is an interface defined in the Kotlin standard library, and declares methods `next()`, which should give the next element of the collection, and `hasNext()`, which returns a boolean indicating whether the collection has more elements.

Providing iterators that conform to `Iterator` lets you loop over both collections in a standard way using the `in` infix function:

```
val cars = listOf("Lamborghini", "Ferrari", "Rolls-Royce")
val numbers = mapOf("Brady" to 12, "Manning" to 18, "Brees" to 9)

for (car in cars) {
    println(car)
}
for (qb in numbers) {
    println("${qb.key} wears ${qb.value}")
}
```

Even though `cars` is a list and `numbers` is a map, you use the same approach to iterate through them, thanks to `Iterator`.

There is a subtle distinction for the iterators of the list and map types. The `List` type provides an iterator by conforming to `Collection`, which then conforms to another interface named `Iterable`. In comparison, the `Map` type has an iterator due to a `Map` extension function.

Comparable

`Comparable` declares an operator function used to compare an instance to other instances.

```
public interface Comparable<in T> {
    public operator fun compareTo(other: T): Int
}
```

Suppose you want to create a `Boat` class and compare boat sizes, with each boat conforming to a `SizedVehicle` interface:

```
interface SizedVehicle {
    var length: Int
}
```

You can make Boat implement SizedVehicle and also conform to Comparable:

```
class Boat: SizedVehicle, Comparable<Boat> {  
    override var length: Int = 0  
    override fun compareTo(other: Boat): Int {  
        return when {  
            length > other.length -> 1  
            length == other.length -> 0  
            else -> -1  
        }  
    }  
}
```

The implementation of compareTo returns an Int indicating the relative size of two boats based on their lengths.

Note: The meaning of each returned Int might not feel intuitive. For some help, here is the documentation for this interface method. compareTo:

“Returns zero if this object is equal to the specified other object, a negative number if it’s less than other, or a positive number if it’s greater than other.”

You can then compare the sizes of two boats using operators such as >:

```
val titanic = Boat()  
titanic.length = 883  
  
val qe2 = Boat()  
qe2.length = 963  
  
println(titanic > qe2) // > false
```

Challenges

Create a collection of interfaces for tasks at a pet shop that has dogs, cats, fish and birds.

The pet shop duties can be broken down into these tasks:

- All pets need to be fed.
- Pets that can fly need to be caged.
- Pets that can swim need to be put in a tank.

- Pets that walk need exercise.
 - Tanks and cages need to occasionally be cleaned.
1. Create classes for each animal and adopt the appropriate interfaces. Feel free to simply use a `println()` statement for the method implementations.
 2. Create homogeneous arrays for animals that need to be fed, caged, cleaned, walked, and tanked. Add the appropriate animals to these arrays. The arrays should be declared using the interface as the element type, for example `var caged: Array<Cageable>`.
 3. Write loops that will perform the proper tasks (such as feed, cage, walk) on each element of each array.

Remember, at any time you can look at the challenge solutions for this chapter for help.

Key points

- Interfaces define a contract that classes, objects, and other custom types can **implement**.
- By implementing an interface, a type is required to **conform** to the interface by implementing all methods and properties of the interface.
- A type can implement any number of interfaces, which allows for a quasi-multiple inheritance not permitted through subclassing.
- The Kotlin standard library uses interfaces extensively. You can use many of them, such as `Comparable`, on your own types.

Where to go from here?

Interfaces help you decouple behavior from implementation. Since interfaces are types themselves, you can still declare an array of `Vehicle` instances. The array could then contain bicycles, trucks, or cars. In addition, bicycles could be enumerations and trucks could be classes! But every `Vehicle` has a particular set of properties and methods you know you must implement.

In the next chapter, you'll learn more about a topic that's been briefly mentioned in earlier chapters: The use of **generics** in defining Kotlin types.